Project Guideline

English Premier League Database Application • COSC 457

This guideline is a working blueprint for planning, building, documenting, and packaging the English Premier League (EPL) database application. It aligns the proposed project with the class expectations shown in the supplied guideline deck: report content, Java Swing implementation, manuals, and executable delivery.

1. Project Vision

The project will simulate the business operations of a professional football league environment. The database application will manage clubs, players, managers, stadiums, fixtures, match results, match events, transfers, standings, and selected commercial relationships. The final software package should feel like a practical business system rather than a simple classroom demo.

1.1 Objectives

- Design a realistic relational database model that supports league operations from season setup to reporting.
- Implement the user interface in Java Swing with JDBC connectivity to MySQL.
- Produce meaningful SQL reports for operational and analytical use.

- Deliver complete documentation: design report, reference manual, tutorial manual, and software package.

2. Scope of the Application

The recommended minimum scope is intentionally broad enough to support a strong ERD and SQL reporting, but narrow enough to complete within a semester project. The scope can be expanded with sponsor tracking, attendance, or broadcast contracts only after the core workflow is stable.

Module	In Scope	Out of Scope / Optional Later
Season Management	season creation, matchweeks, active season status	multi-league support
Club Management	club profile, manager, stadium, city	academy teams
Player Management	player master data, positions, club registration	agent contracts
Fixture Management	schedule, venue, match date, status	automated fixture generation engine
Result Processing	scores, winner, match events, standings update	VAR decision logs
Transfers	incoming/outgoing transfers, fee, date, history	loan clauses and installments
Reports	standings, top scorers, roster views, fixtures by week	predictive analytics

3. Recommended Deliverables

- Database design report with introduction, target business, business process, user requirements, ERD, and Gantt chart.
- Java source code and executable JAR file for the Swing application.
- Database script containing CREATE TABLE statements and INSERT statements for sample data.
- Reference manual with functions explained in alphabetical order and accompanied by screenshots.
- Tutorial manual organized by work scenarios and step-by-step walkthroughs.
- Promotional flyer or one-page application summary if required by the instructor.

4. Team Roles and Suggested Task Split

Role	Primary Responsibility	Suggested Outputs
Project coordinator	schedule, integration, final packaging	timeline, checklist, final review
Database designer	ERD, schema, keys, sample data	ERD, SQL script, data dictionary
Application developer	Swing forms, JDBC, validations	screens, controllers, executable JAR

Documentation lead	report and manuals	report, tutorial, reference manual
QA / testing support	test cases, bug tracking, screenshots	test log, corrected screenshots

5. Development Standards

5.1 Database Standards

- Use singular table names or a consistent plural naming style; do not mix naming patterns.
- Primary keys should be simple, stable identifiers such as club_id, player_id, fixture_id, and season_id.
- Foreign keys must be enforced and named consistently.
- Use lookup tables for positions, event types, and status values when repeated values appear across records.
- Capture history rather than overwrite critical transfer or registration information.

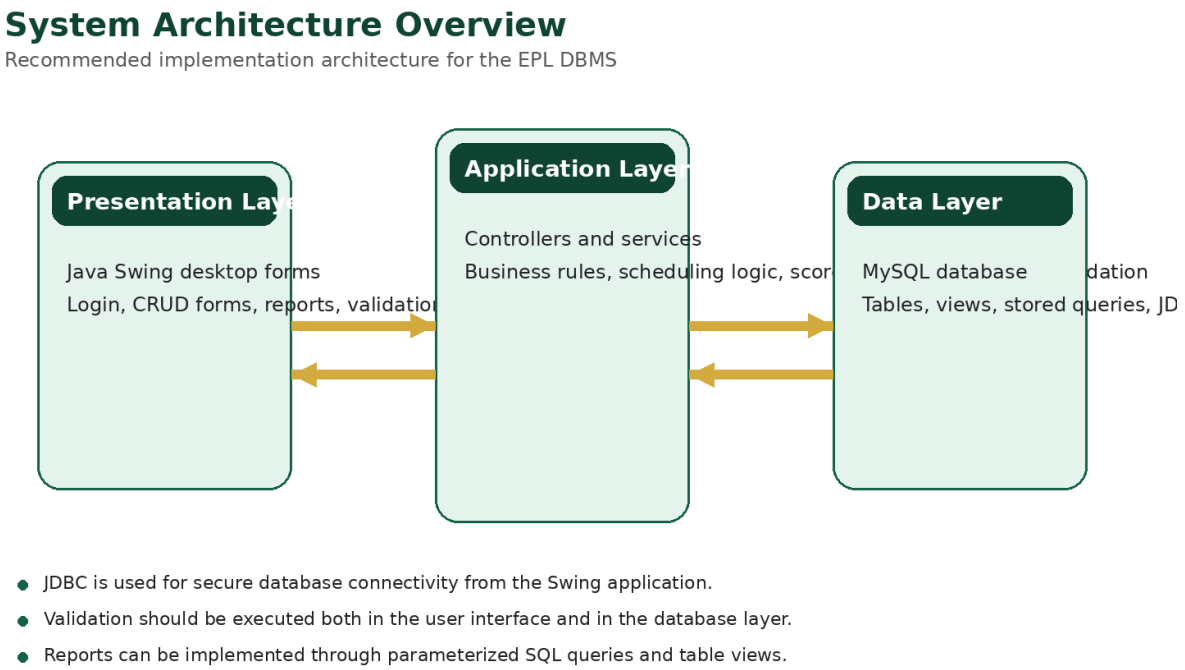
5.2 Java Swing Standards

- Use a consistent menu structure: Dashboard, Clubs, Players, Fixtures, Results, Transfers, Reports, Admin.
- Place validation messages near the relevant form controls and avoid silent failures.
- Every input screen should have clear Save, Reset, Search, and Back/Close actions where appropriate.
- Use JTable for listings and search results; avoid free-form outputs for tabular data.
- Use prepared statements for all database operations to prevent SQL injection and formatting errors.

5.3 Documentation Standards

- Use one naming convention across all documents, schema objects, and screens.
- Every manual screenshot should include a short caption that explains what the user is seeing.
- Replace prototype screenshots with live application screenshots before final submission if the software is complete.
- When describing a function, include purpose, required inputs, actions, validation, and output.

6. Core Screens to Implement



Recommended three-layer implementation view.

Screen	Purpose	Priority
Login	authenticate administrator or reporting user	High
Dashboard	show quick statistics and navigation	High
Club Management	create, update, and browse clubs	High
Player Registration	manage player master data and season registration	High
Fixture Scheduler	schedule matches by season and matchweek	High
Result Entry	record score and match events	High
Transfer Management	maintain transfer history	Medium
Standings / Reports	run queries and analytical outputs	High

7. Testing and Quality Checklist

- Does every table load without runtime errors?
- Are all required fields validated before save?
- Do foreign key lookups prevent invalid club-player or fixture relationships?
- Do result updates refresh standings correctly?
- Do reports show the expected data for a selected season or matchweek?
- Can the application connect to the database from the required environment?

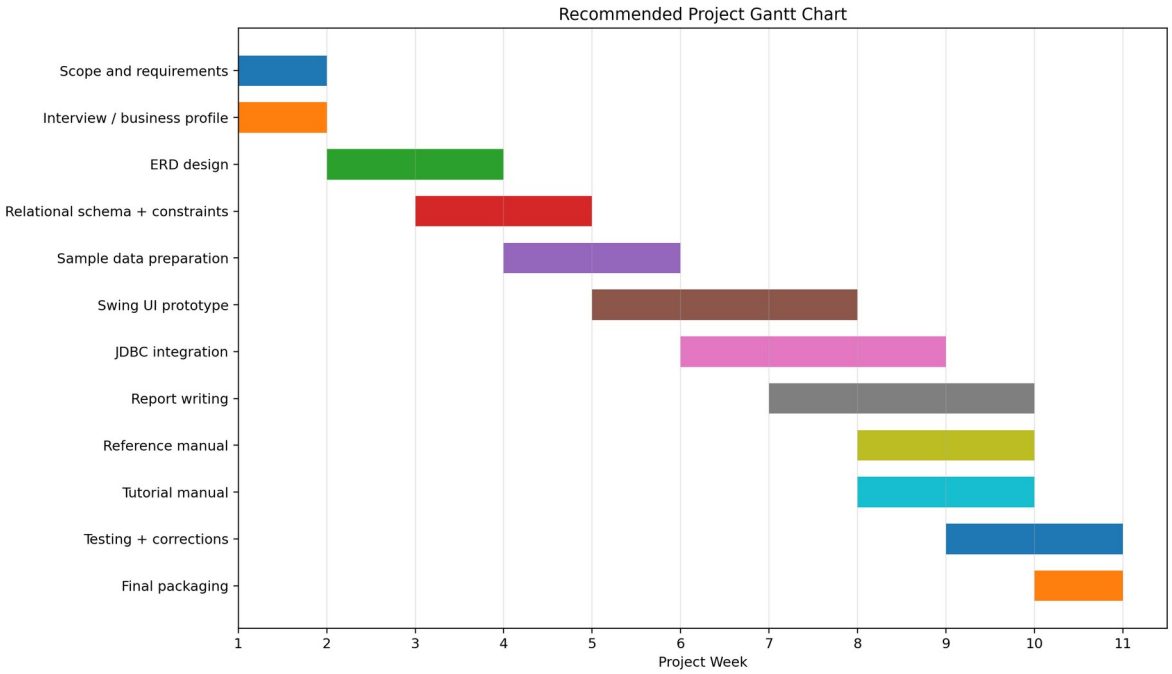
8. Risk Log and Mitigation

Risk	Impact	Mitigation
Scope becomes too large	unfinished features and weak testing	freeze the minimum viable scope early
Inconsistent sample data	queries return incorrect results	assign one owner for data seeding
Late documentation	rushed manuals and missing screenshots	capture screenshots during each completed screen
Database connection problems	application fails during demo	test JDBC connection early and keep local fallback setup
Weak integration between modules	broken workflows	merge work frequently and run scenario-based tests

9. Submission Checklist

1. Confirm that the report matches the actual design and features implemented in the application.
2. Export and test the executable JAR on a clean environment if possible.
3. Include the database creation and sample data script.
4. Review the reference manual for alphabetical function ordering.
5. Review the tutorial manual for realistic user scenarios and step-by-step screenshots.
6. Perform a final spelling, layout, and screenshot audit across all deliverables.

10. Recommended Weekly Plan



Recommended development timeline for the semester project.

This guideline is intended to keep the team focused on a realistic build path. It should be updated whenever the team changes scope, adds a module, or reorganizes the division of work.